Providing a Baseline for Human Activity Recognition Systems under Structured Sensor Corruption

JP Medina
University of Toronto
jpmedinag@cs.toronto.edu

Jadyn Wall
University of Toronto
jadyn@cs.toronto.edu

Yujie Zhu
University of Toronto
iceyujie.zhu@mail.utoronto.ca

Abstract

This is an abstract.

Code is available in our public repository [1].

1 Introduction

Human Activity Recognition (HAR) is a very active area of research due to technological advancements of smartphones, microelectronics, and sensors. This is the purpose of *Ubiquitous sensing*, a research area focused of collecting data acquired from sensors and utilizing it to extract information [2]. The recognition of human activities has many uses in healthcare, monitoring, and human-computer interaction [2]. Reyes-Ortiz et al. [3] provided the Human Activity Recognition Using Smartphones (UCI HAR) dataset, a benchmark built from accelerometer and gyroscope signals collected from 30 subjects performing activities. Subsequent deep learning approaches have matched or exceeded their results on clean data [4].

Despite the strong performance, real-world sensor systems regularly produce degraded data. Sensors age, drift, and miscalibrate, for example: wireless transmission introduces packet loss [5]; battery depletion or physical damage can cause intermittent failure [5]. The resulting corruption spans a broad range of failures that go beyond simple missing data, including constant bias offsets, gradual temporal drift, amplification/attenuation gain shifts, etc. Understanding how HAR models respond to these different forms of degradation is important for building reliable activity recognition systems that generalize beyond clean data.

A number of works have investigated the problem of missing sensor data in HAR [6, 7, 8], demonstrating that random dropout can substaintially degrade classification accuracy. However these works focus on missing sensor data only, missing other sensor fault types. To the best of our knowledge, no prior work has provided a systematic study of how structured sensor corruption, varying by type, severity, and granularity (sensor modality) affects HAR classifier performance.

In this work, we address this gap by introducing a structured corruption framework defined along three dimensions: corruption type, granularity, and severity. Our framework applies six sensor fault types: stochastic noise, dropout, bias, gain (amplification, and attenuation), and drift, at controlled severity levels and at granularities ranging from individual signal axes, to full sensor modalities. We apply this framework to the UCI HAR dataset, training four classifiers on clean and corrupted data, and evaluating each under systematic corruption. We thus provide the first baseline for understanding how different categories of sensor corruption affect activity recognition performance.

2 Related Work

HAR using wearable inertial sensors is a research area with well-established benchmark datasets and models. Lara and Labrador [2] provided a survey on the research area, covering sensors used, feature extraction methods for time series based datasets, learning models typically used, and evaluation metrics. The UCI HAR dataset [3] introduced by Reyes-Ortiz et al. [3] provided a multiclass SVM baseline achieving 96% accuracy. Deep learning methods used by Ronao et al. [4] provided similar results with 94.79% classification accuracy on the raw sensor data, and 95.75% classification accuracy if the featured frequency information is included. Saha et al. [8] provided another baseline using a lightweight LSTM model with KNN imputation, achieving, 93.89% accuracy. These results confirm that HAR on clean data is a well-studied problem.

Missing sensor readings is the most common studied problem when regarding non-clean data assumptions in sensor produced data. Adhikari et al. [9] provided a comprehensive survey on imputation of missing data in internet of things. (add one more sentence) In the context of HAR, Hossain et al. [6] provided a study on sensor-based HAR with missing data by evaluating the effect of randomly missing accelerometer data, finding that accuracy rates decreased significantly even with small missing data rates. Hossain et al. [7] later introduced the idea of explicitly inducing missing data in the training sets to make models more resistant to missing data and showed that recognition accuracy significantly increased without requiring imputation. Saha et al. [8] addressed the same problem on the UCI HAR dataset, using KNN imputation to recover missing readings. Xaviar et al. [10] proposed a cleaning module: a multimodal fusion model with a denoising autoencoder that learns to correct missing input signals before classification.

These works represent important contributions in sensor degraded data, however they suffer from the same limitation of only considering sensor dropout, and thus study imputation as a solution to this problem. However many sensor fault types cannot be simply solved by imputation as they do not represent missing data.

Sensor corruption literature has been studied extensively. Ni et al. [5] provided a taxonomy of sensor network data faults drawn from real deployments: stuck-at-faults, offset faults, calibration faults, noise, drift, etc. Li et al. [11] extended this in a review of sensor fault diagnosis techniques, distinguishing between incipient failures that are difficult to detect early (bias, drift, gain) and abrupt failures such as sudden noise spikes or hardware disconnection. The discussion of sensor degraded data on HAR datasets have largely been focused on abrupt failures, for which data imputation can fill the gaps, however incipient failures ...

We base our sensor corruption framework on the physical sensor faults introduced by Li et al. [11]. We focus on sensor bias, drift, gain (amplification, attenuation), noise, and dropout which correspond to a documented physical failure mode. Thus we ensure our framework and corruption scenarios we study are not arbitrary but reflect degradation patterns that real HAR deployments will encounter.

3 Methodology

3.1 Dataset

Our investigation was focused on the Human Activity Recognition using Smartphones dataset from Reyes-Ortiz et al. [3]. The dataset provided was already split between training and test data. The dataset contains the raw signals as well as a dataset containing 561 features that were extracted from each signal window using the frequency domain.

Due to the feature dataset removing the time component of the signal, we decided against using the feature dataset because manipulating the featured signals would not be possible. Our study focuses on the raw signal data, therefore we applied our corruption framework directly on the raw signal data. Note that it would be possible to reconstruct the frequency-based feature dataset after a corruption is applied by performing the feature engineering outlined in their work. However that is beyond the scope of this study, we simply decided to work against the raw signal data itself and compare against clean data.

3.2 Data Representation

The dataset consists of multivariate time-series sensor data with shape (N, T, C), where N is the number of samples, T is the number of timesteps per window, and C is the number of sensor channels.

Since classical machine learning models require fixed-length feature vectors, we transform each sample into a one-dimensional representation by flattening across the temporal and channel dimensions:

$$X_{\text{flat}} \in \mathbb{R}^{N \times (T \cdot C)}.$$

The channel set consists of two sensor modalities with three axes each: body accelerometer (x, y, z) and gyroscope (x, y, z).

3.3 Corruption Framework

The corruption framework is built along three dimensions: *Type*, *Granularity*, and *Severity*.

3.3.1 Corruption Type

We investigated various different corruption types, each designed to mimic a distinct sensor failure mode:

1. **Stochastic noise:** Adds random noise to the signal window,

   $$X' = X + \lambda\, \sigma(X)\, \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1).$$

   This simulates stochastic sensor noise.
2. **Dropout:** Randomly zeros out a fraction of timesteps in the signal window. Simulates intermittent sensor failure or signal loss.
3. **Bias:** Adds a constant offset to the signal window proportional to the channel's standard deviation,

   $$X' = X + \lambda\, \sigma(X).$$

   This simulates miscalibration or sensor aging.
4. **Gain:** Scales the signal by a constant multiplier,

   $$X' = \lambda X,$$

   where amplification ($\lambda > 1$) simulates sensor sensitivity amplification and attenuation ($0 < \lambda < 1$) simulates sensor sensitivity degradation.
5. **Drift:** Adds a linearly growing offset over the signal window,

   $$X'_t = X_t + d_t\, \sigma(X), \quad d_t = \text{linspace}(0, \lambda, T).$$

   This simulates gradual sensor drift over time.

3.3.2 Granularity

Corruption can be applied at different levels of granularity by specifying which channels are affected by the corruption. The dataset contains 6 channels across 2 sensors. Body accelerometer (x, y, z) and gyroscope (x, y, z). This allows corruption to be applied to an entire sensor modality, a single axis within a modality, or any combination, enabling targeted investigation.

Note that although possible, we decided against investigating applying multiple corruption types to the data at a time. The reason for this being that we wanted to targeted investigation of corruption types, mixing corruption types would make harder to attribute a corruption type.

3.3.3 Severity

Each corruption type has an associated severity parameter that controls the degree of corruption applied. The meaning of severity is type-specific.

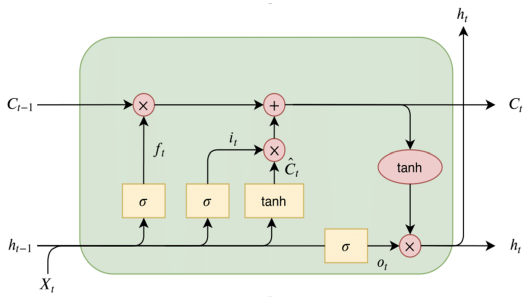

Figure 1: LSTM Network Architecture. Reproduced from [8].

1. Stochastic: Corresponds to the fraction of channel's standard deviation used as the noise scale.
2. Dropout: Corresponds to the fraction of timesteps zeroed out in the window.
3. Bias: Corresponds to a fraction of the channel's standard deviation added as a constant offset.
4. Gain: Corresponds to a constant multiplier applied to the signal.
5. Drift: Corresponds to a multiplier of the channel's standard deviation by the end of the signal window.

3.4 Models

We evaluate three classical machine learning models: K-Nearest Neighbors (KNN), Logistic Regression, and Support Vector Machine (SVM), along with one deep learning model: Long Short-Term Memory (LSTM).

3.4.1 K-Nearest Neighbors (KNN)

We use a KNN classifier with $k = 5$ and Euclidean distance. The model operates directly on flattened feature vectors without additional preprocessing.

3.4.2 Logistic Regression

We implement Logistic Regression using a pipeline that includes feature standardization followed by a linear probabilistic classifier:

$$\text{Pipeline} = \text{StandardScaler} \rightarrow \text{LogisticRegression}$$

We use the `lbfgs` solver with a maximum of 3000 iterations to ensure convergence.

3.4.3 Support Vector Machine (SVM)

We use a linear SVM (LinearSVC) combined with feature standardization:

$$\text{Pipeline} = \text{StandardScaler} \rightarrow \text{LinearSVC}$$

The maximum number of iterations is set to 5000 to ensure stable training.

3.4.4 Long Short-Term Memory (LSTM)

We implement the LSTM framework described by Saha et al. [8] in their investigation of the effects of missing sensor data on activity classification using the UCI HAR dataset, as shown in Figure 1.

Unlike the flattened inputs used with the classical models, each sample is represented as $X_n = [x_1, x_2, ..., x_T]$, where $x_t \in \mathbb{R}^C$ is the sensor data at time t, with C sensor channels.

At each timestep, the LSTM updates the hidden state h_t and the cell state c_t with the following equations:

1. Forget Gate:
$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$
2. Input Gate:
$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$
3. Output Gate:
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$
4. Candidate Cell state:
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$
5. Cell State Update:
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
6. Hidden State Update:
$$h_t = o_t \odot \tanh(c_t)$$

Where σ is the sigmoid activation function, $tanh$ is the hypberbolic tangent activation function, and $\odot$ is the element-wise multiplication operation.

To replicate the LSTM implemented in [8], we use a single-layer LSTM with 128 hidden units and batch-first inputs; the final hidden state is passed through dropout ($p = 0.2$), a dense layer with 64 units and ReLU activation, and a final linear classifier over the six activity classes; the model uses cross-entropy loss and the Adam optimizer with a learning rate of 10^{-3}. The model is trained for 80 epochs with a batch size of 64 using a 20% validation split and the checkpoint with the best validation accuracy is selected.

3.5 Training and Evaluation Protocol

All models are trained on clean training data and evaluated on both clean and corrupted test sets. Reusing the clean-trained model across corruption levels isolates the effect of test-time sensor degradation.

We report:

- Accuracy
- Macro-F_1 score
- Confusion Matrix (for selected experiments)

3.6 Experimental Setup

For each corruption type, we vary the severity parameter across predefined ranges and evaluate model performance on:

- Accelerometer channels (ACCL)
- Gyroscope channels (GYRO)

This allows us to analyze:

- Cross-model robustness (KNN vs Logistic vs SVM vs LSTM)
- Modality-specific sensitivity (ACCL vs GYRO)
- Performance degradation trends across corruption severity

  This design ensures that corruption is isolated to specific modalities, allowing controlled analysis of modality-specific robustness.

4 Experiments

All models are trained on the train/test split provided by the UCI HAR dataset. We performed various experiments to investigate the effects of our corruption framework on the models trained on the dataset. To characterize how each corruption type affects performance, we sweep each type across its severity levels and report accuracy and macro-F_1 as a function of severity. We evaluate all four models under two granularity conditions: accelerometer modality only, and gyroscope modality only. This is done to study the effect of the corruption on different modalities to observe the more sensitive modality to corruption.

5 Evaluation & Results

5.1 Evaluation Metrics

5.2 Results

Baseline Performance. As mentioned in our Methodology section, our corruption framework requires the raw data to perform the various corruption transformations on it. However, the processed data which does not include the time component produces much higher accuracy and macro-F_1 scores due to the fact that many more features are generated. We thus want to establish the baseline performance of the raw data against the processed data to contextualize our results of corruption on the raw data. Figure 2 shows how the processed data achieves much higher performance than the raw data on the classical models. However the LSTM model performs at close to 91.0% accuracy using the raw data. We were able to match Saha et al. [8] with the LSTM model where they achieved 93.89% accuracy. Since this architecture is designed to exploit sequential data, evaluating the LSTM on the processed data is not appropriate.

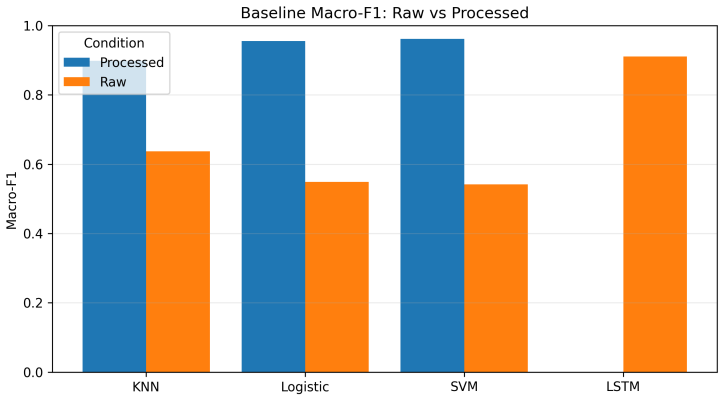

Figure 2: Baseline macro-F_1 score for the raw vs processed data against the various models studied.

Effect of Dropout. Figure 3 outlines the effects of dropout corruption. The macro-F_1 score decreases significantly only for the KNN model, while for the other models (logistic, SVM, LSTM) the macro-F_1 score decreases steadily. This supports our earlier point that dropout corruption is currently the only corruption type that is being studied in literature and does not significantly decrease accuracy or macro-F_1 score except up to the point that dropout corruption removes test data.

Effect of Drift. Figure 4 outlines the effects of drift corruption. This corruption type significantly affects the models. As severity increases, we can observe that the macro-F_1 score significantly decreases for all models. LSTM is the model most negatively affected by the corruption. The reason why drift corruption has a more negative impact than the dropout corruption is because drift corruption corrupts every timestep in the signal window with a monotonically increasing offset. This fundamentally alters the properties of the signal, it no longer preserves the integrity of underlying signal making it increasingly unrecognizable to models that rely on learned temporal features. This

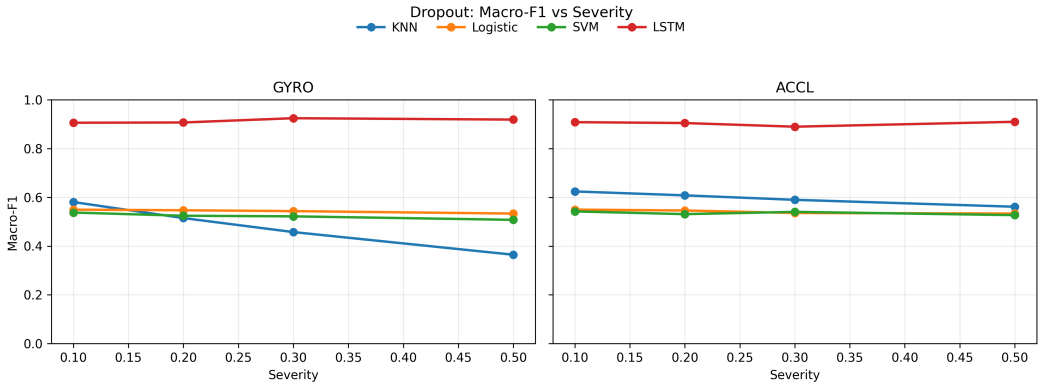

Figure 3: Macro-F_1 score for the dropout corruption on the test dataset on the various models.

result outlines the significance of training models that are resistant to additional corruption types beyond dropout as drift corruption can heavily affect activity recognition systems.

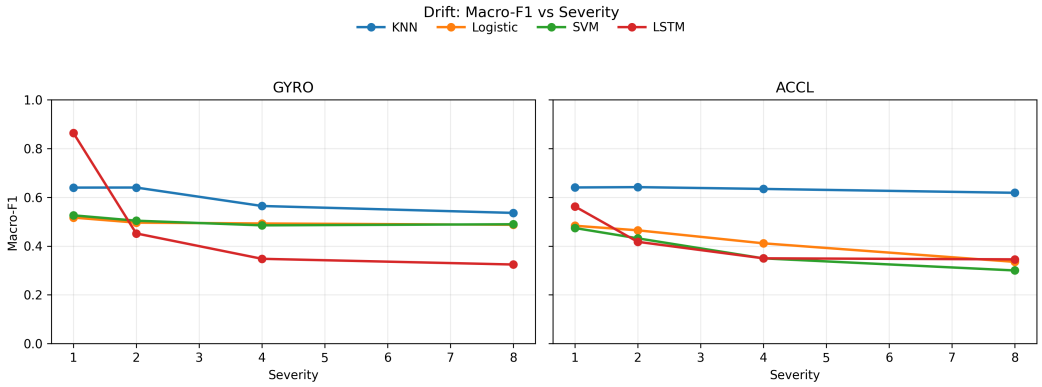

Figure 4: Macro-F_1 score for the drift corruption on the test dataset on the various models.

Effect of Gain. Gain corruption is separated into amplification (gain > 1) and attenuation (gain < 1). Note that gain $= 1$ is the baseline performance without any corruption.

Figure 5 outlines the effects when gain < 1. As the severity decreases (the corruption becomes more significant), we can observe that the macro-F_1 score decreases for some models. We can observe that the logistic, SVM, and LSTM models are largely unaffected by attenuation. The only model significantly affected by attenuation is KNN. The reason is that attenuation scales the test signal values down uniformly, moving them away from the region of the feature space occupied by the clean training data. KNN relies on distances to training examples, so this shift causes test points to be matched against incorrect neighbors. Logistic regression, SVM, and LSTM are less affected because they learn decision boundaries that are more invariant to uniform scaling of the input.

Figure 6 outlines the effects when gain > 1. Amplification scales the test signal values up uniformly, moving test points further away from the origin in the feature space. Logistic regression and SVM are largely unaffected because their decision boundaries are defined by the direction of feature differences rather than their absolute magnitude, making them inherently more invariant to uniform scaling. Surprisingly, KNN macro-F_1 scores increase as severity increases, which is the inverse of the attenuation effect. A plausible explanation is that the clean training and test data are not perfectly aligned in scale. Amplification may move test samples closer to their true class cluster in the training set. This is an unexpected finding that should be investigated in future work.

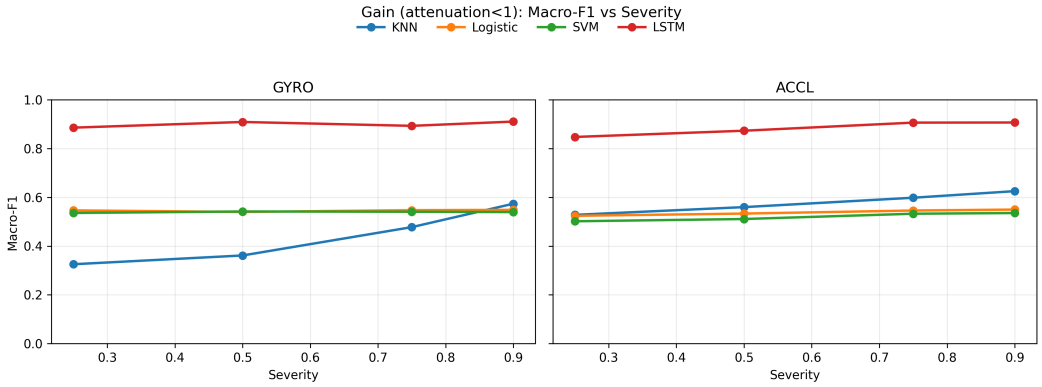

Figure 5: Macro-F_1 score for the gain corruption (attenuation) on the test dataset on the various models.

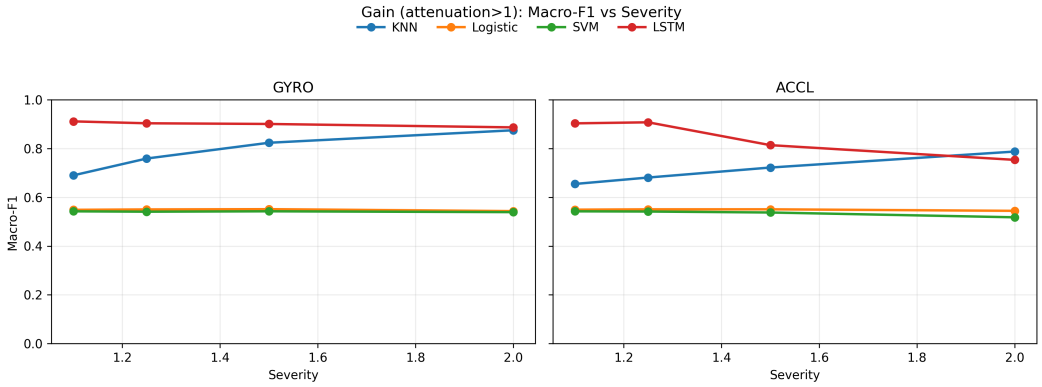

Figure 6: Macro-F_1 score for the gain corruption (amplification) on the test dataset on the various models.

Effect of Bias. Figure 7 outlines the effects of bias corruption. This corruption type also significantly affects the models. As severity increases, we can observe that the macro-F_1 score decreases significantly for all models. Similarly with drift corruption, LSTM is the model most affected by the bias corruption. The reason why bias corruption has a significant impact is because bias corruption shifts the entire signal by a constant offset, moving it outside the distribution the models were trained on. A constant offset displaces every timestep, which causes the model's learned feature representations to no longer correspond to the corrupted input. Again, this highlights the importance of training models robust to additional corruption types beyond dropout.

Effect of Stochastic Noise. Stochastic noise had minimal impact on performance, this is because stochastic noise only corrupts a subset of timesteps within the signal window and only displaces these timesteps by a small random amount scaled to the signals standard deviation. Since the noise is applied independently for each timestep corrupted, it tends to average out over the window, leaving the overall signal structure largely intact. Models trained on clean data are typically already robust to this type of perturbation as the corrupted signal remains similar to what was seen during training.

Modality Study.

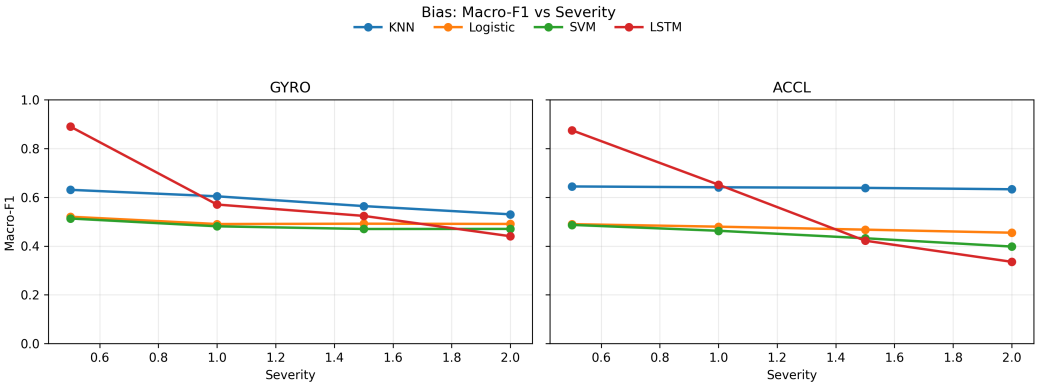

Figure 7: Macro-F_1 score for the bias corruption on the test dataset on the various models.

6 Conclusion

6.1 Future Work

1. Evaluate effect of training on corrupted data
2. Combine multiple corruption types
3. Evaluate model performance on processed corrupted data

References

[1] JP Medina, Jadyn Wall, and Yujie Zhu. sensor-corruption. `https://github.com/jpmedinagl/sensor-corruption`, 2026.

[2] Oscar D. Lara and Miguel A. Labrador. A survey on human activity recognition using wearable sensors. *IEEE Communications Surveys & Tutorials*, 15(3):1192–1209, Nov 2013. doi: https://doi.org/10.1109/surv.2012.110112.00192.

[3] Jorge Reyes-Ortiz, Davide Anguita, Alessandro Ghio, Luca Oneto, and Xavier Parra. Human Activity Recognition Using Smartphones. UCI Machine Learning Repository, 2013. DOI: https://doi.org/10.24432/C54S4K.

[4] Charissa Ann Ronao and Sung-Bae Cho. Human activity recognition with smartphone sensors using deep learning neural networks. *Expert Systems with Applications*, 59:235–244, 2016. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2016.04.032. URL `https://www.sciencedirect.com/science/article/pii/S0957417416302056`.

[5] Kevin Ni, Nithya Ramanathan, Mohamed Nabil Chehade, Laura Balzano, Sheela Nair, Sadaf Zahedi, Eddie Kohler, Greg Pottie, Mark Hansen, and Mani Srivastava. Sensor network data fault types. *ACM Transactions on Sensor Networks*, 5(3):1–29, May 2009. doi: https://doi.org/10.1145/1525856.1525863.

[6] Tahera Hossain, Hiroki Goto, Md Atiqur Rahman Ahad, and Sozo Inoue. A study on sensor-based activity recognition having missing data. *2018 Joint 7th International Conference on Informatics, Electronics & Vision (ICIEV) and 2018 2nd International Conference on Imaging, Vision & Pattern Recognition (icIVPR)*, pages 556–561, Jun 2018. doi: https://doi.org/10.1109/iciev.2018.8641012.

[7] Tahera Hossain, Md. Atiqur Ahad, and Sozo Inoue. A method for sensor-based activity recognition in missing data scenario. *Sensors*, 20(14):3811, Jul 2020. doi: https://doi.org/10.3390/s20143811.

[8] Debashish Saha, Piyush Malik, and Adrika Saha. Activity and subject detection for uci har dataset with & without missing sensor data, 2025. URL https://arxiv.org/abs/2505.06730.

[9] Deepak Adhikari, Wei Jiang, Jinyu Zhan, Zhiyuan He, Danda B. Rawat, Uwe Aickelin, and Hadi A. Khorshidi. A comprehensive survey on imputation of missing data in internet of things. *ACM Computing Surveys*, 55(7):1–38, Dec 2022. doi: https://doi.org/10.1145/3533381.

[10] Sanju Xaviar, Xin Yang, and Omid Ardakanian. Robust multimodal fusion for human activity recognition, 2023. URL https://arxiv.org/abs/2303.04636.

[11] Daoliang Li, Ying Wang, Jinxing Wang, Cong Wang, and Yanqing Duan. Recent advances in sensor fault diagnosis: A review. *Sensors and Actuators A: Physical*, 309:111990, Jul 2020. doi: https://doi.org/10.1016/j.sna.2020.111990.

A Roles

JP. Designed and implemented sensor corruption framework.

Jadyn. Implemented LSTM.

Yujie. Implemented classical machine learning models (KNN, logistic regression, SVM).